

PROVA TÉCNICA

Analista de Qualidade Pleno

Prazo
08.06 - 17hrs

Nível
Pleno

Sistema
API Biblioteca Digital

INSTRUÇÕES GERAIS

- **Prazo de entrega:** Até 08.06.2026 até 17hrs
- **Envio:** Encaminhe todos os artefatos por e-mail genteegestao.recrutamento@t2mlab.com com o Título: NOME COMPLETO – PROVA TÉCNICA QA PLENO

ENTREGAVEIS ESPERADOS

- Documento de Estratégia de Testes (Word ou PDF)
- Planilha Excel com plano de testes estruturado
- Collection Postman (.json) + Environment (.json)
- Código de automação + README com instruções de execução
- Relatório de bugs encontrados (se houver)
- Respostas as questões situacionais

CONTEXTO DO SISTEMA - BIBLIOTECA DIGITAL API

Você recebeu a seguinte solicitação do Product Owner:

"Como usuário da Biblioteca Digital, desejo poder registrar nosso catálogo de livros para que seja possível realizar a gestão de atividades, usuários, livros e outros dados referente aos livros. Esta entrega inicial será apenas dos serviços API sem integração com front-end."

Módulos do Sistema

Controller / Recurso	Descrição
Users	Gestão de usuários do sistema
Activities	Gestão de atividades para bibliotecários
Authors	Gestao de autores
Books	FOCO PRINCIPAL - Gestão do catálogo de livros
CoverPhotos	Gestao de capas de livros

API de Referência

URL Base: <https://fakerestapi.azurewebsites.net>

Documentação Swagger: <https://fakerestapi.azurewebsites.net/index.html>

Verbos HTTP disponíveis para Books Controller: GET (listar / buscar por ID), POST (criar), PUT (atualizar), DELETE (remover)

PARTE 1 - ANÁLISE DE REQUISITOS E PLANEJAMENTO ESTRATÉGICO

Teórica

1.1 - Análise Crítica dos Requisitos

Com base na solicitação do usuário e na documentação Swagger disponível, responda:

a) Identificação de Lacunas e Ambiguidades

- Liste pelo menos 4 lacunas ou ambiguidades nos requisitos fornecidos.
- Para cada lacuna: qual é o problema, qual o impacto potencial, como deveria ser esclarecida.
- Considere: regras de negócio não definidas, validações ausentes, comportamentos de erro não especificados, relacionamentos entre entidades não documentados.

b) Perguntas Críticas ao PO / Time de Desenvolvimento

- Elabore pelo menos 5 perguntas críticas que você faria antes de iniciar os testes.
- Classifique cada pergunta por impacto: Alto / Médio / Baixo.
- Justifique por que a resposta é relevante para o planejamento de testes.

c) Mapeamento de Riscos

- Identifique os principais riscos técnicos e de negócio associados a entrega.
- Para cada risco: Probabilidade (Alta/Média/Baixa), Impacto (Alto/Médio/Baixo) e Mitigação proposta.
- Considere riscos de integração, segurança, performance, dados e disponibilidade.

1.2 - Estratégia de Testes

Elabore um documento de estratégia de testes contemplando os itens abaixo:

- **Escopo:** O que será testado e o que está fora do escopo. Justifique as exclusões.
- **Abordagem:** Quais tipos de testes serão realizados (funcional, contrato/schema, negativos, segurança, performance). Explique a escolha.
- **Priorização:** Como você prioriza os testes? Quais áreas são mais críticas e por quê?
- **Ambiente e Dados:** Configurações de ambiente necessárias, variáveis, dependências e estratégia de dados de teste.
- **Crêterios de Entrada e Saida:** Condições para início e encerramento dos ciclos de teste.
- **Métricas de Qualidade:** Defina ao menos 5 métricas que serão monitoradas. Para cada uma: como será coletada e qual o threshold aceitável.

1.3 - Análise de Impacto e Integrações

- Mapeie as dependências entre Books e os demais recursos (Authors, CoverPhotos, Activities, Users).
- Identifique cenários que envolvam mais de um recurso simultaneamente (testes de integração).
- Descreva o impacto de falhas na Controller Books nos demais módulos.

- Proponha um mapa de dependências (descritivo ou em formato de tabela).

PARTE 2 - PLANEJAMENTO DE TESTES FUNCIONAIS

Prática

Elabore o planejamento completo de testes para a Controller Books, cobrindo todos os verbos HTTP disponíveis. Os casos de teste devem ser documentados em planilha Excel conforme o formato especificado abaixo.

2.1 - Formato da Planilha de Casos de Teste

Campo	Descrição	Exemplo
ID	Identificador único	TC_BOOKS_001
Cenário / Título	Descrição resumida do que está sendo testado	GET /Books - Listar todos com sucesso
Pré-condições	Estado necessário antes da execução	API disponível; ao menos 1 livro cadastrado
Passos	Passo a passo detalhado	1. Enviar GET /api/v1/Books 2. Verificar status 3. Validar schema
Resultado Esperado	O que deve acontecer	Status 200; array de objetos Book; schema valido
Resultado Obtido	Preenchido durante execução	(preencher na execução)
Status	Passou / Falhou / Bloqueado	(preencher na execução)
Prioridade	Alta / Media / Baixa	Alta
Tipo de Teste	Funcional / Contrato / Segurança / Performance / Integração / Negativo	Funcional
Evidência	Screenshot, log, response copiado	(preencher na execução)
Observações	Notas, bugs relacionados	Verificar paginação se aplicável

2.2 - Categorias de Teste Obrigatórias

O plano DEVE cobrir, no mínimo, as seguintes categorias para cada endpoint da Controller Books:

- **Testes Funcionais:** CRUD completo: criar, listar todos, buscar por ID, atualizar, deletar. Validar status codes, headers e corpo da resposta.
- **Testes de Contrato / Schema:** Validar estrutura dos objetos retornados conforme o Swagger. Verificar tipos de dados, campos obrigatórios, nulos e formatos.
- **Testes de Borda (Boundary):** Valores no limite de campos numéricos, strings vazias vs. máximo de caracteres, datas invalidas, IDs no limite do tipo.
- **Testes Negativos:** IDs inexistentes, payloads malformados, campos obrigatórios ausentes, tipos de dados incorretos.
- **Testes de Segurança Básica:** SQL Injection, XSS básico via campos de texto, acesso sem autenticação (se aplicável), manipulação de IDs (IDOR).
- **Testes de Performance Basica:** Tempo de resposta dentro de thresholds para operações de leitura e escrita. Comportamento com requisições paralelas.
- **Testes de Integração:** Books + Authors (criar livro com autor inexistente), Books + CoverPhotos (remover livro com capa associada).
- **Idempotencia e Consistência:** PUT idempotente? DELETE de recurso já removido retorna 404? GET de recurso recém-criado retorna dados corretos?

2.3 - Requisitos Mínimos de Cobertura

Endpoint	Mínimo de Casos de Teste
GET /Books (listar todos)	Mínimo 3 casos
GET /Books/{id} (buscar por ID)	Mínimo 3 casos
POST /Books (criar livro)	Mínimo 3 casos
PUT /Books/{id} (atualizar livro)	Mínimo 3 casos
DELETE /Books/{id} (remover livro)	Mínimo 2 casos

PARTE 3 - EXECUÇÃO DE TESTES DE API COM POSTMAN

Prática

3.1 - Configuração da Collection

Crie uma Collection no Postman chamada "Biblioteca Digital - Books API" com as especificações:

- **Organização:** Pastas por verbo HTTP (GET, POST, PUT, DELETE) ou por fluxo (Smoke Tests, Functional, Negative, Contract).
- **Environment:** Crie "BibliotecaDigital_ENV" com variáveis: baseUrl, bookId (dinâmico), lastCreatedId (populado por script).

3.2 - Relatório de Bugs

Caso sejam encontradas falhas, documente cada bug.

PARTE 3 - AUTOMAÇÃO DE TESTES

Prática + Justificativa Técnica

4.1 - Escolha e Justificativa do Framework

Escolha um framework de automação de testes de API e justifique tecnicamente a escolha.

A justificativa deve abordar:

- Por que este framework é adequado para o contexto (API REST, equipe, CI/CD)?
- Quais são as limitações do framework escolhido?
- Como o framework se integra ao pipeline de CI/CD?
- Quais alternativas foram consideradas e por que foram descartadas?

4.2 - Implementação dos Testes Automatizados

Implemente, no mínimo, os seguintes testes automatizados:

- **Teste 1 (Obrigatório - Fluxo E2E):** Criar livro -> Verificar criação via GET -> Atualizar título -> Verificar atualização -> Deletar -> Verificar retorno 404.
- **Teste 2 (Obrigatório - Schema):** Validação de schema completa: todos os campos do objeto Book, tipos, formatos e obrigatoriedade.
- **Teste 3 (Obrigatórios - Negativos):** Ao menos 3 cenários negativos automatizados (ID inválido, payload incompleto, tipo de dado incorreto).
- **Teste 4 (Diferencial - Parametrizado):** Validar múltiplos IDs ou payloads com data-driven testing.

- **Teste 5 (Diferencial - Concorrência):** Simular múltiplas requisições simultâneas e validar consistência dos dados.

4.3 - README e Documentação

O pacote de automação deve conter um README.md com:

- Pré-requisitos (dependências, versões necessárias)
- Instruções de instalação (passo a passo)
- Como executar os testes (comando único preferencialmente)
- Como interpretar os relatórios gerados
- Estrutura de pastas e arquivos explicada
- Como adicionar novos casos de teste ao projeto

4.4 - Proposta de Arquitetura de Automação Escalável

Descreva uma proposta de arquitetura de automação escalável para o projeto completo (não apenas Books), considerando:

- Estrutura de pastas e organização do código
- Padrão de design (ex: Builder Pattern para payloads, Client Layer para chamadas HTTP)
- Separação de configurações por ambiente (dev, staging, prod)
- Estratégia de dados de teste (fixtures, factories, massa de dados)
- Integração com CI/CD (Jenkins, GitHub Actions, GitLab CI)
- Geração de relatórios (Allure, HTML Report, notificações Slack)
- Critérios para novos testes integrarem o repositório (code review, padrões mínimos, cobertura)

PARTE 5 - QUESTÕES SITUACIONAIS

Teórica - Avaliação Comportamental e Senioridade

Responda as questões abaixo de forma objetiva e com profundidade técnica. Não há resposta única correta - o avaliador buscara consistência de raciocínio, maturidade profissional e domínio da área.

Questão 1 - Requisito Ambíguo

Durante o refinamento, você identifica que no requisito "o sistema deve validar o livro antes de salvar" está completamente vago. O PO está indisponível por 2 dias e o desenvolvimento já começou assumindo uma interpretação. Qual é sua postura? Como você garante que os testes serão relevantes sem ter clareza total do comportamento esperado? Quais riscos você documenta?

Sua resposta:

Questão 2 - Pressão por Release

Faltam 4 horas para o deploy em produção de uma feature crítica. Você identificou 2 bugs classificados como "Alto": um bug funcional que afeta 30% dos cenários de busca e um bug de schema que expõem dados parcialmente incorretos. O Product Owner pressiona para liberar, argumentando que "nenhum usuário real vai perceber isso agora". Qual é sua posição? Como você conduz essa conversa? O que você documenta?

Sua resposta:

Questão 3 - Mentoria de Júnior

Um colega testador júnior está responsável por testar um endpoint complexo (PATCH com regras de negócio elaboradas). Após 2 dias, ele entrega apenas 5 casos básicos, todos passando, sem nenhum caso negativo ou de borda - e está empolgado com o resultado. Como você aborda essa situação? Como você o mentoreia sem desmotivar? Que exercício prático você proporia para desenvolver o pensamento analítico dele?

Sua resposta:

Questão 4 - Falsos Positivos na Automação

A suite de automação de regressão no pipeline de CI/CD apresenta 15% de taxa de falsos positivos (testes que falham sem bug real). Isso gerou desconfiança no time e alguns desenvolvedores começaram a ignorar falhas no pipeline. O que você faz imediatamente? Como você investiga e trata os falsos positivos? Como você recupera a credibilidade da suite?

Sua resposta:

Questão 5 - Cobertura Insuficiente em Área Crítica

As métricas mostram que o modulo de autenticação/autorização tem apenas 12% de cobertura automatizada, enquanto o catálogo de livros tem 78%. A sprint está na metade e o backlog está cheio. Como você prioriza? Qual argumento você usa para o time e para o PO para justificar o investimento em cobertura de autenticação? Como equilibra velocidade de entrega com qualidade?

Sua resposta:

Questão 6 - Conflito Dev x QA sobre Severidade

Você registrou um bug como "Crítico": ao deletar um livro com capa associada, a capa permanece como registro órfão no banco (sem FK válida). O desenvolvedor argumenta que é "cosmético" e deveria ser "Baixo", pois "não aparece para o usuário". A discussão está ficando acalorada na

reunião. Como você fundamenta sua avaliação de severidade? Como você conduz a mediação? Quais critérios objetivos você utiliza para classificar severidade de bugs?

Sua resposta:
